Review 1

Problem, Literature and Proposed Idea

IntentOS: A Userspace Intent Kernel that Schedules, Authorises, and Recovers User Goals as First-Class Processes

Proposed system report. This document describes the intended IntentOS design, not a line-by-line account of the present prototype.

Research papers were located through Google Scholar and rewritten in original wording. Patent records were read on Google Patents. The patent discussion is an academic novelty statement, not a legal opinion.

1. Project Title

IntentOS: A userspace intent kernel that schedules, authorises, and recovers user goals as first-class processes.

2. Problem Statement

A person preparing for a meeting still opens Calendar, then Files, then Maps, then Mail, and copies facts from one window into another. The host operating system already owns the files, the network, and the processes that those applications use. It has no record that those separate actions belong to one goal. Because the goal is not a system object, nothing schedules it against other goals, nothing limits it to only the resources that goal needs, and nothing can resume it from the last finished step if messaging fails halfway through.

Existing assistants can answer a sentence or call one service. They treat the request as a one-shot dialogue. When the dialogue ends, the goal disappears. Competing goals are not ordered by deadline. A failed step starts the whole request again. Privilege is the privilege of the signed-in user or of the installed application, not the privilege of this particular goal.

The problem is therefore not the absence of calendar or mail software. The problem is the absence of a managed execution entity for the goal itself.

3. Motivation

People describe work as outcomes: prepare for the 4 PM meeting, gather the related files, tell the group, and leave on time. Computers still require the person to choose applications and to carry state between them. That manual transfer is slow, easy to get wrong, and invisible to the scheduler.

Operating systems already solve a similar problem for programs. A process has an identity, a state, a priority, a set of resources, a protection domain, and a way to be restarted from saved state. Those mechanisms stop at the application boundary. Two goals that both need the calendar are not two processes the scheduler can see. A meeting goal that only needs to read the calendar and send one message still runs with whatever the mail application is allowed to do.

IntentOS is motivated by moving that discipline up one level. The unit of management becomes what the user wants, while the host operating system continues to manage real processes, files, and memory. The layer stays in userspace. It does not replace Linux or Windows.

4. Objectives

1. Represent each natural-language goal as an Intent Process with identity, lifecycle state, priority, deadline, resource needs, permission grant, plan, and checkpoint.
2. Turn the goal into an ordered sequence of capability calls, such as read calendar, find files, compute a route, and prepare a message.
3. Admit that process into a userspace kernel and authorise every required resource before execution, using least privilege for that goal alone.
4. Schedule competing goals by priority and deadline, so a nearer deadline is not ignored merely because another goal arrived first.
5. Execute each step through a service registry and record a checkpoint when the step completes.
6. On failure, resume from the last completed checkpoint instead of repeating finished steps.
7. Keep external effects, such as sending mail or creating a calendar event, explicit and recoverable to the extent the outside service allows.
8. Leave the host kernel unchanged and use its files, processes, and network as the execution substrate.

5. Existing System and Existing Approaches

Application-centred desktops. The current environment is a set of independent programs. The operating system schedules those programs and protects their address spaces. It does not know that a calendar lookup and a later email are one piece of work. Recovery is whatever each application implements for itself. If the user closes a window, the goal is gone.

Digital assistants. Commercial assistants accept speech or text, infer an intent, and run a task flow that calls calendar, maps, or messaging services [P1]–[P5]. They are effective for a single command. The intent is a dialogue object inside the assistant, not a process the operating system can queue, age, or checkpoint beside other goals. Permission is usually the user’s account or an application grant, not a fresh least-privilege set computed for that sentence.

Context-aware applications. Context-aware systems adapt behaviour to location, nearby devices, and changing situation [8], [9]. They improve relevance. They do not admit the user’s goal as a schedulable entity with its own resource table.

Intent-based networking. Intent-based and intent-driven networks let an operator state a desired network outcome and then translate, activate, and assure that outcome in a closed loop [4], [5], [15]. The idea of stating an outcome rather than a procedure is relevant. Those systems manage network configuration, not a person’s meeting, files, and messages, and they do not use a process control block for the user goal.

Language-model agents. Recent models can interleave reasoning with tool calls [12], decide when to call an API [13], and keep a memory of past actions [14]. They show that a goal can be pursued in steps. A typical agent run is still a session. It is not admitted with a deadline, it is not ordered against other user goals, and a crash usually means the session is lost rather than restored at the last committed step.

Classical operating-system mechanisms. Deadline scheduling, capability protection, least privilege, deadlock treatment, and checkpoint recovery are mature for programs and distributed computations [1]–[3], [6], [7], [10], [11]. They are applied to threads, processes, and messages. They are not applied, as a single design, to the natural-language goal that caused those programs to be opened.

6. Literature Survey

Liu and Layland studied how a single processor should serve periodic work that must finish by a deadline [1]. Their deadline-driven rule gives priority to the task whose current deadline is nearest, and they showed when that rule can use the processor fully. IntentOS borrows the ordering idea, but the tasks are user goals with one-off deadlines, not periodic control loops, and the scheduler runs in userspace, so it cannot offer the hard guarantees assumed for dedicated real-time processors.

Sha, Rajkumar, and Lehoczky showed that ordinary locks can cause priority inversion, where a high-priority job waits without bound behind a low-priority one, and they gave inheritance and ceiling rules that bound that wait and avoid certain deadlocks [10]. Any kernel that lets several goals share a calendar or a file service has the same structural risk. The proposed design therefore treats shared services as named resources and acquires them in a way that does not hold one resource while waiting for another.

Coffman, Elphick, and Shoshani surveyed prevention, avoidance, detection, and recovery for deadlock in multiprogrammed systems [7]. Their conditions, including hold-and-wait, explain why a goal that locks the file service and then waits for the network can stall every other goal. IntentOS uses that analysis to keep each execution step to one resource acquisition.

Dennis and Van Horn defined protection and naming for separate computations that share objects [6]. A computation should name only the objects it is allowed to use. Saltzer and Schroeder later stated least privilege directly: every program and user should run with only the privileges needed for the job, so that a mistake or a compromise has a limited blast radius [2]. IntentOS applies that rule to the goal. A meeting-preparation goal may read the calendar and send one message. It does not inherit a general right to every file the user can open.

Roesner and colleagues argued that modern platforms isolate applications, then grant them too much, too early, through install-time manifests or prompts the user learns to ignore [11]. They attach permission to an authentic user action at the moment of use. IntentOS is in the same family, with a different principal. The principal is the goal, and the grant is computed from the resources that goal’s plan actually names.

Elnozahy, Alvisi, Wang, and Johnson classified rollback recovery into checkpoint-based and log-based families [3]. A checkpoint restores a saved state. A log records the nondeterministic events needed to replay past that state. Their survey also makes clear that saved state and the outside world are not one atomic transaction. IntentOS checkpoints each completed capability. If a later step fails, finished steps stay done. If an external send may already have happened, the design must ask the user to look at the destination rather than pretend the send can be rolled back.

Schilit, Adams, and Want defined context-aware applications that react to the person’s changing surroundings, including proximate selection and context-triggered actions [8]. Dey then gave an operational definition of context and a toolkit that separates sensing from interpretation [9]. A meeting goal needs that kind of situation: time, place, related files, and participants. Context alone does not create a process, a deadline queue, or a permission domain.

Leivadeas and Falkner describe intent-based networking as a closed loop: express an intent, translate it, resolve conflicts, activate it, and assure that the network still matches the intent [4]. Pang, Yang, Chen, Song, and Guizani survey the same shift for self-driving networks, where application intent is separated from low-level configuration [5]. Jacobs and colleagues showed that operators can state network intent in natural language and have it refined into configuration [15]. These results support “say the outcome, then compile it into controlled actions.” They compile into network state. IntentOS compiles into a lifecycle for a personal goal.

Yao and colleagues showed that a language model can alternate between a reasoning trace and an external action, and that the action results keep the plan honest [12]. Schick and colleagues showed that a model can learn when a tool such as a calendar or a search API is worth calling [13]. Park and colleagues showed that a stored record of experience, summarised over time, lets an agent continue a plan instead of reacting only to the latest sentence [14]. IntentOS uses this step-by-step use of tools, then places the resulting plan under process management so the run survives beyond one model session.

Stoess argued that microkernel systems need scheduling policy that a user or subsystem can control, rather than one fixed global policy [16]. That supports a userspace scheduler whose policy is about goals. It does not by itself define the goal as the scheduled object.

7. Research Gap

The literature splits into two groups that do not meet.

Operating-system work explains how to schedule, protect, and recover a computation [1]–[3], [6], [7], [10], [11]. The computation is a program or a thread. The user's goal never becomes the record that is queued.

Assistant, context, and agent work explains how to interpret a sentence, call services, and remember a dialogue [8], [9], [12]–[14], together with the assistant patents [P1]–[P7]. The run is a session. When it ends or crashes, there is no process-table entry from which another goal can be ordered, and there is no per-goal privilege smaller than the user's login.

Intent-based networking shows that an outcome can be translated, activated, and assured [4], [5], [15]. The outcome is network behaviour, not a personal task that spans calendar, files, maps, and mail, and the assurance loop is not a process checkpoint.

No surveyed approach does all of the following together:

• admit a natural-language goal as a durable process with priority and deadline,
• derive a least-privilege grant from that goal's own plan,
• order that goal against other pending goals,
• dispatch only named capabilities, and
• resume after failure from the last completed step of that same goal.

That combination is the gap IntentOS is proposed to fill.

8. Proposed Solution

IntentOS adds one userspace kernel above the host operating system. The user states a goal. The kernel understands it, plans it, admits it, authorises it, schedules it, executes it, and recovers it.

Understand. The sentence is parsed into a structured intent. "Prepare everything for my 4 PM meeting" becomes a meeting-preparation intent whose time is 16:00, not a free-form chat reply.

Plan. The kernel produces an ordered capability sequence. For the meeting example the sequence is: read the calendar event at 16:00, find related files, compute the route to the room, and prepare a message to the project group. The plan is data attached to the process. It is not a hidden script inside one application.

Admit. An Intent Process is created. It receives a unique identity, the goal text, a priority, an absolute deadline, the resource list taken from the plan, an empty result set, and the state NEW. This record is the process control block of the goal.

Authorise. Before any step runs, the permission manager checks each required resource against the goal's policy. The meeting goal is allowed to read the calendar, read files, use maps, and write a message. It is not allowed to read unrelated stores, change system settings, or call a service the plan never named. A step that fails this check is blocked. Later steps are not automatically given a wider grant.

Schedule and run. Runnable goals move to READY. The scheduler chooses the next goal by deadline and priority, with an aging rule so a low-priority goal cannot wait forever. The chosen goal runs one step, through the service registry, against the real calendar, file, map, or messaging service. When the step returns, its result is stored and a checkpoint is committed. The process returns to READY if more steps remain.

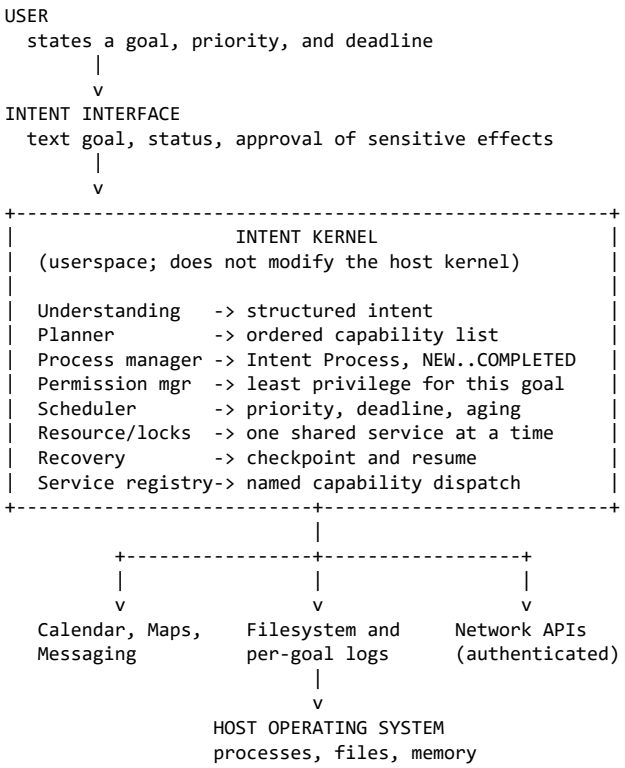
Complete or recover. If every step succeeds, the process enters COMPLETED and reports the meeting, the files, the leave-by time, and the prepared notice. If the messaging step fails, steps one to three remain committed. Recovery puts the process back to READY at step four only. The calendar is not read again and the route is not computed again.

External services cannot always undo a send. The recovery rule is therefore: completed local steps are not repeated; an uncertain external effect is reported to the user before it is retried.

Worked outcome. Ten manual steps across four applications become one admitted process. The process was only ever allowed to touch calendar, files, maps, and messaging. Its identity, deadline, and checkpoints remain visible while other goals wait in the same queue.

9. Proposed Architecture

The user states a goal. The Intent Kernel decides what the goal needs, whether it is allowed, when it should run, and how to recover it if a step fails. The host operating system underneath is left unchanged.



Kernel part	Responsibility
Process manager	Creates the Intent Process, stores its control block, and moves it through NEW, READY, RUNNING, WAITING, RECOVERY, COMPLETED, and FAILED.
Planner	Binds the structured intent to an ordered list of capabilities and records that list on the process.
Permission manager	Grants only the resources named by the plan. Default for an unknown service is deny until the user confirms.
Scheduler	Orders READY goals by near deadline, then priority, and applies aging so waiting goals make progress.
Resource and lock manager	Serialises use of each shared service so goals do not hold one service while waiting for another.
Recovery manager	Writes a checkpoint after each successful step and restarts a failed goal at the first unfinished step.
Service registry	Looks up a capability and calls the real service. The planner never receives a general shell.

The lifecycle of an Intent Process is:

```
NEW -> READY -> RUNNING -> COMPLETED
                  |
                  +-- resource unavailable -> WAITING -> READY
                  |
                  +-- step failed -> RECOVERY -> READY at last checkpoint
```

10. Patent-Oriented Notes

Records were read on Google Patents. They show that “infer a user intent and complete a task” is already heavily protected. The comparison is about technical difference, not about freedom to operate.

Ref.	Record	Assignee	What it already covers
P1	US10769385B2, granted 2020	Apple	Infer intent from speech, score interpretations, and carry out the chosen meaning.
P2	US20140297284A1, granted as US10475446B2	Apple	Use context such as the calendar or the message on screen to decide which command is meant.
P3	US11269678B2, granted 2022	Apple	Connect a digital assistant to third-party calendar, messaging, and navigation services to finish a task.
P4	US11656884B2, granted 2023	Apple	Map a natural-language intent object onto an installed application and return whether the task succeeded.
P5	US8677377B2, granted 2014	Apple (from SRI)	Build an assistant from active domains and services that can be invoked for a request.
P6	US20200327895A1, later granted as US12087308B2	Apple	A general intelligent assistant that identifies a domain and completes the request inside it.
P7	US10877784B1, granted 2021	Facebook, now Meta	Detect an intent to organise an event from a message thread and write the chosen event to calendars.

Novel features of the proposed system

- The goal is a first-class execution record: identity, state, priority, deadline, resource table, permission grant, and checkpoint, modelled on a process control block.
- Several goals are ordered against each other. The assistant patents complete a request. They do not define a deadline queue of goals.
- Least privilege is attached to the goal’s plan, not only to the installed assistant or the user account.

- Failure resumes that same goal at the last completed capability. It does not open a new dialogue that repeats finished work.
- The service registry is a capability boundary. Understanding and planning do not receive a general command shell.

What is different from existing solutions

Assistants in P1–P7 already parse language, use context, call calendars and maps, and report success. Copying that pipeline would not be new. The difference is the management layer around the pipeline. Intent-based networking [4], [5], [15] already closes a loop around an intent, but the intent is a network outcome. Language-model agents [12]–[14] already act in steps, but the step trace is a session memory, not an operating-system process with admission control. Permission research [2], [11] already limits programs. It does not mint a new principal each time a person states a goal.

Initial statement of what might be protected

The candidate subject matter is the method of admitting a natural-language goal as a userspace process, deriving that process's permission set from its own capability plan, scheduling the process against other admitted goals by deadline and priority, and resuming it from the last checkpointed capability after a partial failure. Protection would be sought for that combination, not for speech recognition, calendar access, or deadline scheduling considered alone. A patent attorney would still need to search the full claim language of P1–P7 and the continuation families behind them before any filing.

11. References

Papers below were identified through Google Scholar and cited from the original publication.

[1] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," Journal of the ACM, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[2] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proceedings of the IEEE, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[3] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, "A survey of rollback-recovery protocols in message-passing systems," ACM Computing Surveys, vol. 34, no. 3, pp. 375–408, Sep. 2002.

[4] A. Leivadeas and M. Falkner, "A survey on intent-based networking," IEEE Communications Surveys & Tutorials, vol. 25, no. 1, pp. 625–655, 2023.

[5] L. Pang, C. Yang, D. Chen, Y. Song, and M. Guizani, "A survey on intent-driven networks," IEEE Access, vol. 8, pp. 22862–22873, 2020.

[6] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," Communications of the ACM, vol. 9, no. 3, pp. 143–155, Mar. 1966.

[7] E. G. Coffman, M. J. Elphick, and A. Shoshani, "System deadlocks," ACM Computing Surveys, vol. 3, no. 2, pp. 67–78, Jun. 1971.

[8] B. Schilit, N. Adams, and R. Want, "Context-aware computing applications," in Proc. 1st Workshop on Mobile Computing Systems and Applications (WMCSA), IEEE, 1994, pp. 85–90.

[9] A. K. Dey, "Understanding and using context," Personal and Ubiquitous Computing, vol. 5, no. 1, pp. 4–7, 2001.

[10] L. Sha, R. Rajkumar, and J. P. Lehoczky, "Priority inheritance protocols: An approach to real-time synchronization," IEEE Transactions on Computers, vol. 39, no. 9, pp. 1175–1185, Sep. 1990.

[11] F. Roesner, T. Kohno, A. Moshchuk, B. Parno, H. J. Wang, and C. Cowan, "User-driven access control: Rethinking permission granting in modern operating systems," in Proc. IEEE Symposium on Security and Privacy, 2012, pp. 224–238.

[12] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in Proc. 11th International Conference on Learning Representations (ICLR), 2023.

[13] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” in Advances in Neural Information Processing Systems, vol. 36, 2023.

[14] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in Proc. 36th Annual ACM Symposium on User Interface Software and Technology (UIST), 2023.

[15] A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, R. A. Ferreira, L. Z. Granville, et al., “Hey, Lumi! Using natural language for intent-based network management,” in Proc. USENIX Annual Technical Conference, 2021, pp. 625–639.

[16] J. Stoess, “Towards effective user-controlled scheduling for microkernel-based systems,” ACM SIGOPS Operating Systems Review, 2007.

Patent records (Google Patents; not counted as research-paper references)

[P1] G. Evermann, “System and method for inferring user intent from speech inputs,” U.S. Patent 10,769,385, Sep. 8, 2020.

[P2] T. R. Gruber et al., “Using context information to facilitate processing of commands in a virtual assistant,” U.S. Patent Application 2014/0297284, Oct. 2, 2014, granted as U.S. Patent 10,475,446.

[P3] T. R. Gruber, C. D. Brigham, A. J. Cheyer, D. Keen, and K. Kocienda, “Systems and methods for integrating third party services with a digital assistant,” U.S. Patent 11,269,678, Mar. 8, 2022.

[P4] C. E. Radebaugh et al., “Application integration with a digital assistant,” U.S. Patent 11,656,884, May 23, 2023.

[P5] A. Cheyer and D. Guzzoni, “Method and apparatus for building an intelligent automated assistant,” U.S. Patent 8,677,377, Mar. 18, 2014.

[P6] T. R. Gruber, A. J. Cheyer, and D. Keen, “Intelligent automated assistant,” U.S. Patent Application 2020/0327895, Oct. 15, 2020, granted as U.S. Patent 12,087,308.

[P7] D. Testuggine et al., “Determining and servicing user intent with a virtual assistant,” U.S. Patent 10,877,784, Dec. 29, 2020.

Sixteen research publications are cited. Reference [16] is cited without volume and page numbers because those were not rechecked against the ACM PDF. Confirm reference [11], pages 224–238, against the IEEE proceedings PDF before submission. The remaining volume, issue, and page data were checked against publisher or author records.